

RideShare Auth Service — HLD & LLD

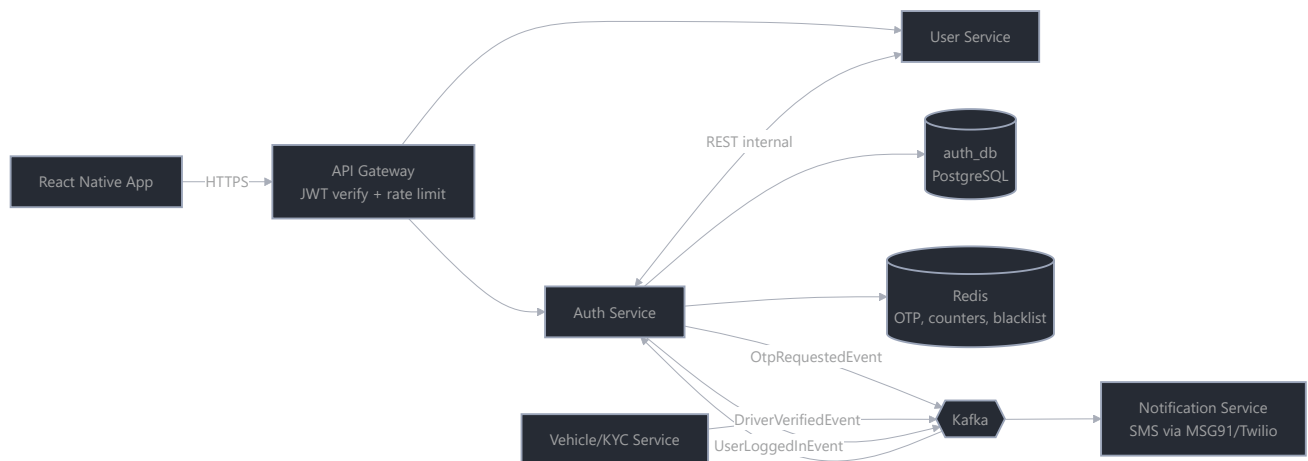
Scope: phone + OTP login/signup, profile completion for new users, JWT session management, and rider/driver mode switching.

1. Key decisions

#	Decision	Reason
1	One account, two roles. Everyone is a RIDER ; DRIVER is unlocked by KYC + verified vehicle.	Same user toggles from one home screen.
2	Auth Service owns identity + sessions . User Service owns profile + active mode .	Keeps auth small; profile fields change often.
3	Access token (JWT, 15 min) carries roles , activeMode , driverVerified .	Gateway/services can authorize without a DB call.
4	Mode switch = POST /auth/mode/switch → new access token, same refresh token. No re-login, no OTP.	Instant toggle; token stays truthful.
5	Refresh token is opaque, hashed in DB, single-use, rotated. Reuse = revoke all sessions of that user.	Theft detection.
6	Users are keyed by UUID; phone is a unique column.	Phone can change without key cascades.

2. HLD

2.1 Components

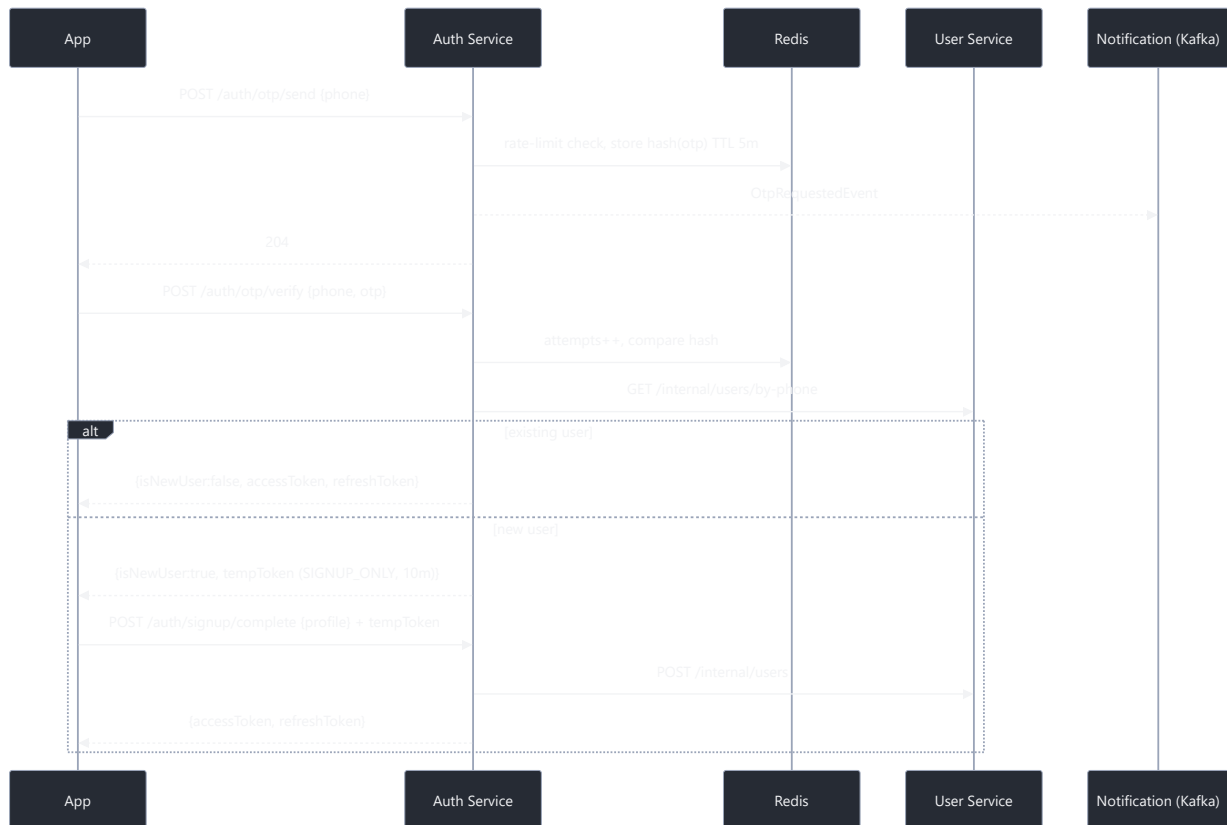


2.2 Responsibilities

- **Auth Service:** OTP issue/verify, token issue/refresh/revoke, signup orchestration, mode switching, device/session tracking.
- **User Service (internal REST):** user CRUD, profile fields, active_mode, GET by-phone.
- **Notification Service:** consumes OtpRequestedEvent, sends SMS. Auth never talks to the SMS provider directly in prod.
- **Gateway:** verifies JWT signature/expiry, rejects SIGNUP_ONLY tokens everywhere except

/auth/signup/complete, forwards X-User-Id, X-Active-Mode, X-Roles headers downstream.

2.3 Login / signup flow



2.4 Mode switch flow

1. App calls `POST /auth/mode/switch {mode: "DRIVER"}` with the current access token.
2. Auth checks the mode is valid for the user's roles (see 3.5), tells User Service to persist `active_mode` (so next login opens on the last-used tab), then issues a **new access token** with updated `activeMode`.
3. App swaps the access token; UI flips tab.
4. If `driverVerified=false`, the switch **succeeds** but the token says so, and the app shows vehicle + license onboarding before offer creation. Ride Service rejects offers without `driverVerified=true`.
5. When KYC approves a vehicle/license, `DriverVerifiedEvent` makes Auth add the `DRIVER verified` flag; it appears in the user's next refreshed token (≤ 15 min) or immediately if the app calls `POST /auth/token/refresh`.

2.5 Security and non-functional

- OTP: 6 digits, SHA-256 hashed in Redis, TTL 5 min, max 5 verify attempts, max 3 sends / 10 min / phone, plus per-IP and per-device limits at the gateway.
- JWT signed RS256 (private key only in Auth; gateway/services use public key via JWKS). HS256 is acceptable for MVP only.
- Logout/ban: refresh token revoked in DB; access token `jti` blacklisted in Redis until expiry.
- Log phone numbers masked (`+91XXXXXX1234`). PII (email, DOB) encrypted at rest in User Service.
- Availability: Auth is stateless, so scale horizontally; Redis and Postgres are the stateful dependencies.
- If User Service is down: login for existing users fails fast with 503 (circuit breaker via Resilience4j).

3. LLD

3.1 Package structure

```
com.rideshare.auth
├─ config      SecurityConfig, OpenApiConfig, AuthProperties (record @ConfigurationProperties)
├─ controller  AuthController, ModeController
├─ dto         request/response records
├─ service     AuthService (orchestrator), OtpService, JwtService,
│              RefreshTokenService, ModeService
├─ client      UserClient (interface) → RestUserClient, StubUserClient
├─ sms         SmsSender (interface) → ConsoleSmsSender, KafkaSmsPublisher
├─ event       OtpRequestedEvent, DriverVerifiedEvent (+ listener)
├─ entity      RefreshToken, OtpLog, AuthSession
├─ repository
└─ exception   ApiException, ErrorCode enum, GlobalExceptionHandler
```

3.2 Database (auth_db)

```
CREATE TABLE auth_sessions (
  session_id  UUID PRIMARY KEY,
  user_id     UUID NOT NULL,
  device_id   VARCHAR(100) NOT NULL,
  device_info VARCHAR(255),
  fcm_token   TEXT,
  ip_address  VARCHAR(45),
  created_at  TIMESTAMP NOT NULL DEFAULT now(),
  last_seen_at TIMESTAMP,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE refresh_tokens (
  token_id    UUID PRIMARY KEY,
  session_id  UUID NOT NULL REFERENCES auth_sessions(session_id),
  user_id     UUID NOT NULL,
  token_hash  VARCHAR(64) NOT NULL UNIQUE,    -- SHA-256 hex
  family_id   UUID NOT NULL,                  -- all rotations of one login share this
  expires_at  TIMESTAMP NOT NULL,
  is_revoked  BOOLEAN NOT NULL DEFAULT FALSE,
  replaced_by UUID,
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

CREATE INDEX idx_rt_user ON refresh_tokens(user_id);
CREATE INDEX idx_rt_family ON refresh_tokens(family_id);

CREATE TABLE otp_logs (
  otp_log_id  UUID PRIMARY KEY,
  phone_number VARCHAR(15) NOT NULL,
  purpose     VARCHAR(10) NOT NULL,          -- LOGIN | SIGNUP
  is_verified BOOLEAN NOT NULL DEFAULT FALSE,
  attempt_count INT NOT NULL DEFAULT 0,
  ip_address  VARCHAR(45),
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

-- local copy of eligibility, updated from DriverVerifiedEvent
CREATE TABLE user_roles (
```

```

    user_id          UUID PRIMARY KEY,
    driver_verified  BOOLEAN NOT NULL DEFAULT FALSE,
    is_banned        BOOLEAN NOT NULL DEFAULT FALSE,
    updated_at       TIMESTAMP NOT NULL DEFAULT now()
);

```

3.3 Redis keys

Key	Value	TTL
otp:{phone}	SHA-256(otp)	300s
otp:attempts:{phone}	verify attempts	300s
otp:sends:{phone}	send count	600s
jwt:blacklist:{jti}	1	remaining token life
signup:used:{tempJti}	1	600s (tempToken is single-use)

3.4 Token model

Access token claims:

```

{
  "sub": "<userId>", "jti": "<uuid>", "scope": "ACCESS",
  "roles": ["RIDER", "DRIVER"],
  "activeMode": "RIDER",
  "driverVerified": false,
  "sid": "<sessionId>",
  "iat": 0, "exp": 0
}

```

Temp token (signup): sub=<phone>, scope=SIGNUP_ONLY, 10 min, single-use.

roles always contains RIDER and DRIVER for every user (anyone may *enter* driver mode);
driverVerified is the real gate for posting offers.

3.5 API contracts

Endpoint	Auth	Request	Success	Errors
POST /auth/otp/send	none	{phoneNumber, deviceId}	204	400 INVALID_PHONE, 429 OTP_RATE_LIMITED
POST /auth/otp/verify	none	{phoneNumber, otp, deviceId, deviceInfo?, fcmToken?}	200 {isNewUser, accessToken?, refreshToken?, tempToken?}	400 OTP_INVALID / OTP_EXPIRED, 429 OTP_ATTEMPTS_EXCEEDED, 403 USER_BANNED
POST /auth/signup/complete	tempToken	{name, email?, gender?, dob?, company?, designation?, workEmail?}	201 token pair	401 TEMP_TOKEN_INVALID, 409 EMAIL_ALREADY_USED, 422 validation

POST /auth/token/refresh	none	{refreshToken}	200 token pair	401 REFRESH_INVALID , 401 REFRESH_REUSED
POST /auth/mode/switch	access	{mode: RIDER DRIVER}	200 {accessToken, activeMode, driverVerified}	400 INVALID_MODE , 403 USER_BANNED
POST /auth/logout	access	{refreshToken}	204	none (idempotent)
POST /auth/logout/all	access	none	204	none
GET /auth/me	access	none	{userId, roles, activeMode, driverVerified}	401

Signup profile fields (stored in User Service):

- Required: name .
- Optional: email , gender , dob , company , designation .
- Later: workEmail , which turns company into a verified trust badge (OTP to the work email domain). Good fit for a commute-pooling product, so keep the column ready.
- Add profilePhoto , emergencyContact via the User Service profile screens, not at signup, to keep onboarding short.

Error body (all endpoints): { "code": "OTP_INVALID", "message": "...", "timestamp": "..." } .

3.6 Core algorithms

verifyOtp(phone, otp)

1. attempts = INCR otp:attempts:{phone} ; if > 5 , delete the OTP and throw OTP_ATTEMPTS_EXCEEDED .
2. Read otp:{phone} ; missing → OTP_EXPIRED ; hash mismatch → OTP_INVALID .
3. On match, delete otp:{phone} and the attempts key (so an OTP can't be reused).
4. Look up the user via UserClient.findByPhone . Existing and active → create session + tokens. Existing but banned → USER_BANNED . Absent → temp token.

refresh(rawToken)

1. hash = sha256(rawToken) ; load row; missing or expired → REFRESH_INVALID .
2. If is_revoked → **reuse detected**: revoke the whole family_id , deactivate the session, throw REFRESH_REUSED .
3. Else mark revoked and set replaced_by ; insert a new row in the same family; build the access token from current user_roles , so eligibility changes propagate here.

switchMode(userId, sid, mode)

1. Validate mode ∈ {RIDER, DRIVER} and the user isn't banned.
2. UserClient.setActiveMode(userId, mode) (idempotent; log failure but still return the token, because the token is the immediate source of truth and User Service reconciles on the next call)

because the token is the immediate source of truth and user service references on the next call.

3. Issue a new access token (new `jti`) with `activeMode=mode` and the current `driverVerified`; blacklist the old `jti`.

3.7 Gateway authorization rules

Route	Rule
<code>/auth/**</code> (except <code>/auth/mode/**</code> , <code>/auth/logout*</code> , <code>/auth/me</code>)	public
<code>/rides/offers</code> (POST)	<code>activeMode=DRIVER</code> and <code>driverVerified=true</code>
<code>/rides/requests</code> (POST)	<code>activeMode=RIDER</code>
everything else	any valid ACCESS token

3.8 Events

- **Publishes:** `OtpRequestedEvent{phone, otp, purpose}` (payload is short-lived, so don't log it), `UserRegisteredEvent{userId}`, `UserLoggedInEvent{userId, deviceId}`.
- **Consumes:** `DriverVerifiedEvent{userId, verified}`, `UserBannedEvent{userId}` (updates `user_roles`, revokes sessions on ban).

3.9 Edge cases to handle

- Verify OTP from a new device while an old session exists: allowed (multi-device); send a "new login" push to the other devices.
- Phone number recycled by a carrier: login works because identity = OTP possession; re-verify sensitive actions in later phases.
- Signup submitted twice (retry): User Service `POST /internal/users` must be idempotent on phone; return the existing user.
- Clock skew: allow 30s leeway on JWT parsing.
- App kill during signup: `tempToken` expires in 10 min; user redoes OTP, no half-created user since the user is only created at `signup/complete`.

4. Build order for this service

1. Migrations (3.2), `AuthProperties`, `ErrorCode`, `GlobalExceptionHandler`.
2. `OtpService` + `ConsoleSmsSender` + `/otp/send` and `/otp/verify` (with `StubUserClient`).
3. `JwtService` + `RefreshTokenService`; `/signup/complete`, `/token/refresh`, `/logout`.
4. `ModeService` + `/mode/switch` and `/me`.
5. Kafka publisher/listener, swap `StubUserClient` for `RestUserClient`.
6. OpenAPI annotations, export `openapi.json`, then Testcontainers tests.

